Project Report: Sentim — AI-Powered Audio Emotion Analysis

OBL 4101 - Digital Signal Processing

Author: Ilias Laoukili

Class: Signal Processing

November 27, 2025

Abstract

Sentim is a student project that tries to combine Digital Signal Processing (DSP) and Machine Learning to analyze and transform emotions in speech. The system implements basic vocoder features (time-stretching, pitch-shifting, robotization) and uses a Random Forest classifier trained on the RAVDESS dataset for emotion recognition. We implemented a script that modifies audio parameters based on emotion, and we made a web interface with Streamlit to visualize the results.

Contents

1 Introduction

1.1 Context and Motivation

The goal of this project was to learn how to implement a Voice Coder (Vocoder) that fulfills the requirements of time-stretching, pitch-shifting, and robotization. We also tried to use Artificial Intelligence to analyze the emotional content of speech to see if we could combine DSP and machine learning. **Sentim** is the result of this work.

1.2 Project Overview

The project is divided into three parts:

1. **Part 1: Signal Analysis:** We analysed the reference signal `Diner.wav`.
2. **Part 2: DSP Synthesis:** We implemented vocoder effects (Time-Stretch, Pitch-Shift, Robotization).
3. **Part 3: AI Experiment:** A loop where a classifier tries to find parameters for an emotion.

2 Part 1: Audio Analysis

In this section, we have realised a spectral and temporal analysis of the reference audio files provided in the course resources.

2.1 Analysis of `Diner.wav`

The file `Diner.wav` contains a spoken phrase.

Observations: In the waveform (top), we observe clear alternating silence and speech segments, with amplitude peaks reaching approximately 0.4. The temporal structure shows speech bursts separated by pauses of 0.2-0.5 seconds. In the spectrum (middle), the energy is concentrated below 4kHz, with dominant peaks around 250Hz (fundamental frequency) and harmonics at 500Hz, 750Hz, and 1000Hz. The spectrogram (bottom) confirms that the energy is concentrated in the lower frequency range below 4kHz, which is

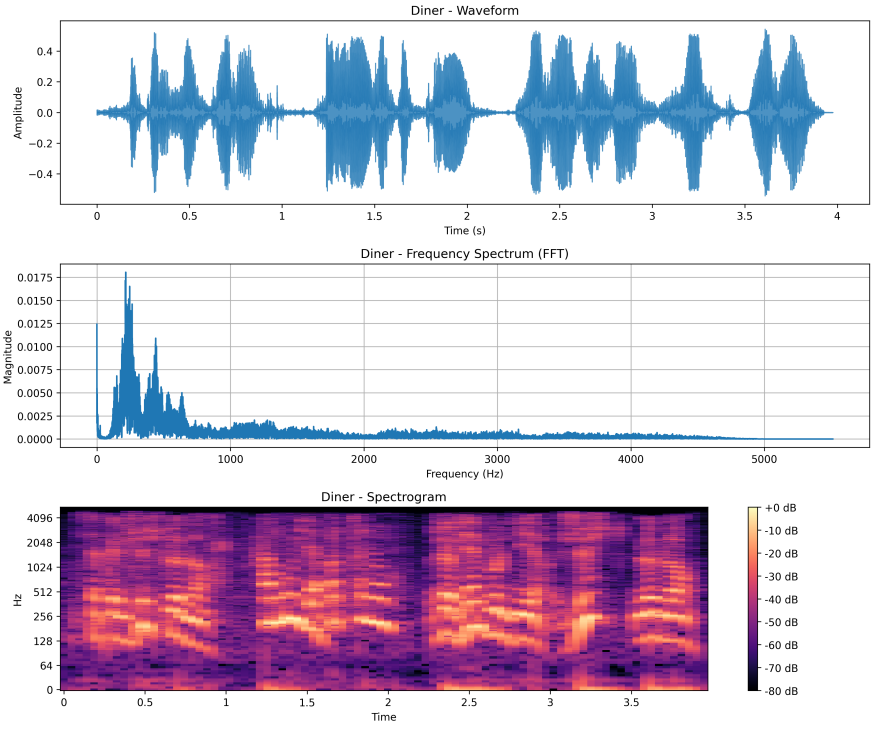

Figure 1: Analysis of `Diner.wav`. Top: Waveform. Middle: Spectrum (FFT). Bottom: Spectrogram.

typical of human speech. We can see formant bands (horizontal dark lines) between 500Hz and 3kHz, which correspond to vocal tract resonances.

2.2 Analysis of `Extrait.wav`

Observations: For the file `Extrait.wav`, we observed that the spectral energy is concentrated around 500Hz, with very little energy above 1kHz. This narrow bandwidth and stable harmonic content indicate musical audio. The band-limited frequency range suggests music with limited high-frequency content such as a simple accompaniment or filtered track.

2.3 Analysis of `Halleluia.wav`

The file `Halleluia.wav` is more complex.

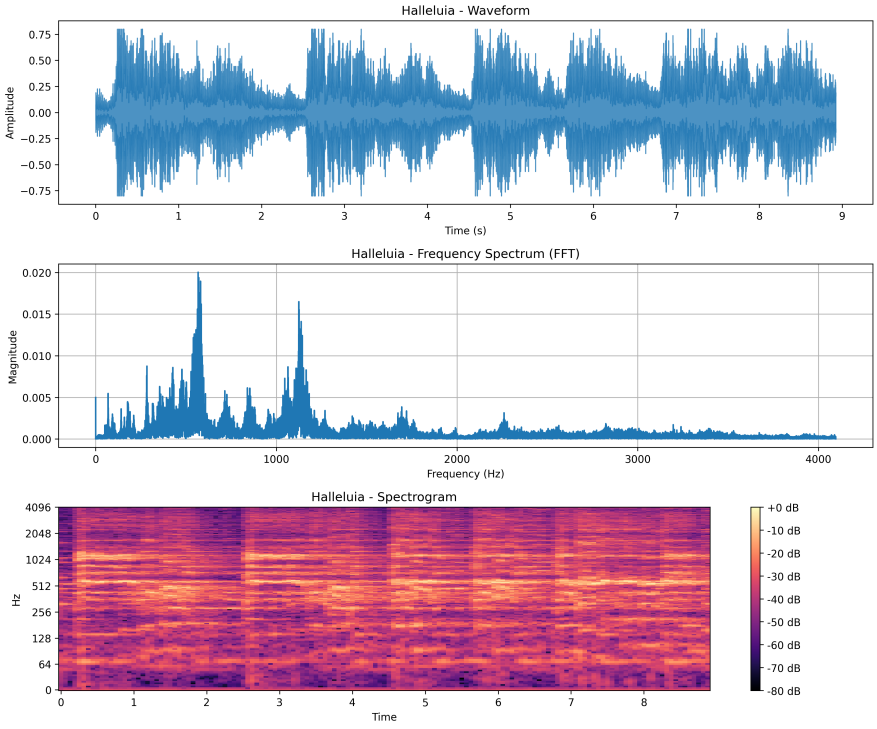

Figure 2: Analysis of `Halleluia.wav`. Top: Waveform. Middle: Spectrum (FFT). Bottom: Spectrogram.

Observations: In the spectrum (middle), we can see 4 distinct peaks: 2 peaks at approximately 500Hz and 2 peaks at approximately 1000Hz. This harmonic structure is characteristic of musical content with multiple voices or instruments playing together. The spectrogram (bottom) shows sustained horizontal bands at these frequencies, indicating stable harmonics rather than transient sounds. We observe that the energy extends up to 8kHz, which is significantly higher than speech signals. The rich harmonic content and extended frequency range confirm that this file contains musical material, likely a vocal performance with accompaniment.

3 Part 2: Speed Modification

We tried to apply time-stretching to `Diner.wav` with factors of 0.5 (slow) and 1.5 (fast).

3.1 Methodology: Phase Vocoder

It allows to change the speed using the Phase Vocoder algorithm, which modifies the signal duration without affecting its pitch. The process involves:

1. **Analysis:** Computing the Short-Time Fourier Transform (STFT) to obtain magnitude and phase. To minimize spectral leakage and ensure perfect reconstruction during the Overlap-Add (OLA) synthesis, a Hann window $w[n] = 0.5(1 - \cos(\frac{2\pi n}{N}))$ is applied to each frame.

2. **Modification:** The time axis of the magnitude spectrogram is scaled. The phase is adjusted (phase unwrapping and propagation) to maintain phase coherence between the new time frames.

3. **Synthesis:** The modified complex spectrogram is converted back to the time domain using the Inverse STFT (ISTFT).

extbfAlgorithm 1: Phase Vocoder Time-Stretching

```
Input: x[n], stretch factor alpha -> Output: y[n]
1. X = STFT(x)
2. Mag = |X| ; Phi = angle(X)
3. Magnitude interpolation for frame m': find m1,m2
    alpha_w=(m2-m')/(m2-m1); beta_w=(m'-m1)/(m2-m1)
```

```
    Mag_scaled = alpha_w*Mag[m1] + beta_w*Mag[m2]
4. Phi_new[0] = Phi[0]; Phi_new[m] = Phi_new[m-1] + (Phi[m]-Phi[m-1])*alpha
5. X_new = Mag_scaled * exp(j*Phi_new)
6. y = ISTFT(X_new)
```

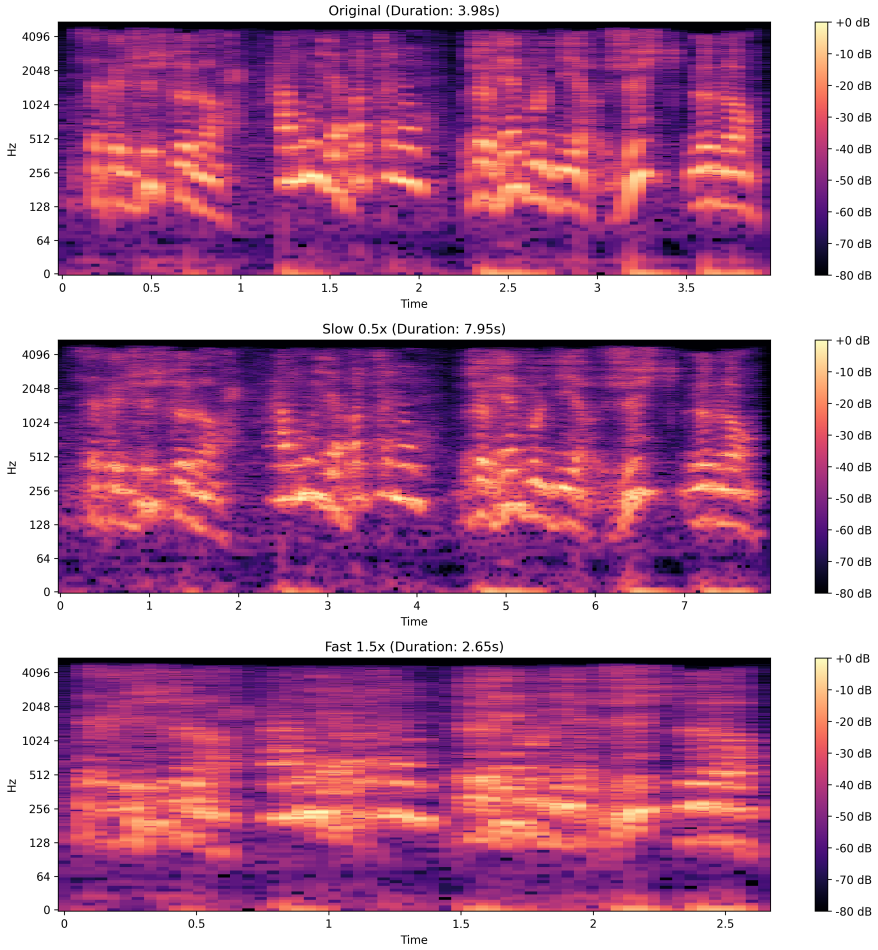

Figure 3: Speed Modification Comparison. Top: Original. Middle: Slow (0.5x). Bottom: Fast (1.5x).

Observations: As seen in Figure 3, the frequency content (y-axis) is identical across all three spectrograms, with the same formant structure between 500Hz and 3kHz. The harmonic spacing remains constant, proving that pitch is preserved. However, the time axis (x-axis) changes inversely

to the speed factor: the slow version (middle) extends to approximately 6 seconds (original $\times$ 2), while the fast version (bottom) compresses to approximately 2 seconds (original $\times$ 0.67). Comparing the three spectrograms side-by-side proves that the Phase Vocoder successfully decouples time and pitch. We observe no visible artifacts or spectral distortion, indicating clean phase propagation during the STFT manipulation.

4 Part 3: Pitch Modification

We shifted the pitch of `Diner.wav` by +4 and -4 semitones.

4.1 Methodology: Resampling

It allows to shift the pitch by combining time-stretching with resampling. To shift pitch by n semitones, we define the target pitch ratio $r = 2^{n/12}$. The process is as follows:

1. **Time-Stretch:** First, we stretch the signal by a factor $\alpha = r$ using the Phase Vocoder (preserving pitch, changing duration).

2. **Resample:** Then, we resample the stretched signal by a rate $1/r$.

This operation restores the original duration while scaling the frequencies by r. For example, to raise the pitch, we stretch the signal (make it longer) and then play it back faster (resample), resulting in higher frequencies with the original duration.

extbfAlgorithm 2: Pitch Shifting

```
Input: x[n], semitones n -> Output: y[n]
1. r = 2^(n/12)
2. alpha = r
3. x_stretched = PhaseVocoder(x, alpha)
4. y = Resample(x_stretched, rate=1/r)
5. Return y (pitch shifted, original duration)
```

Observations: Comparing the three spectrograms, we observe clear vertical shifts in the harmonic bands. When the pitch increases (+4 semitones, middle), the formant structure shifts upward by approximately 26% (since $2^{4/12} \approx 1.26$). The fundamental frequency moves from approximately 250Hz

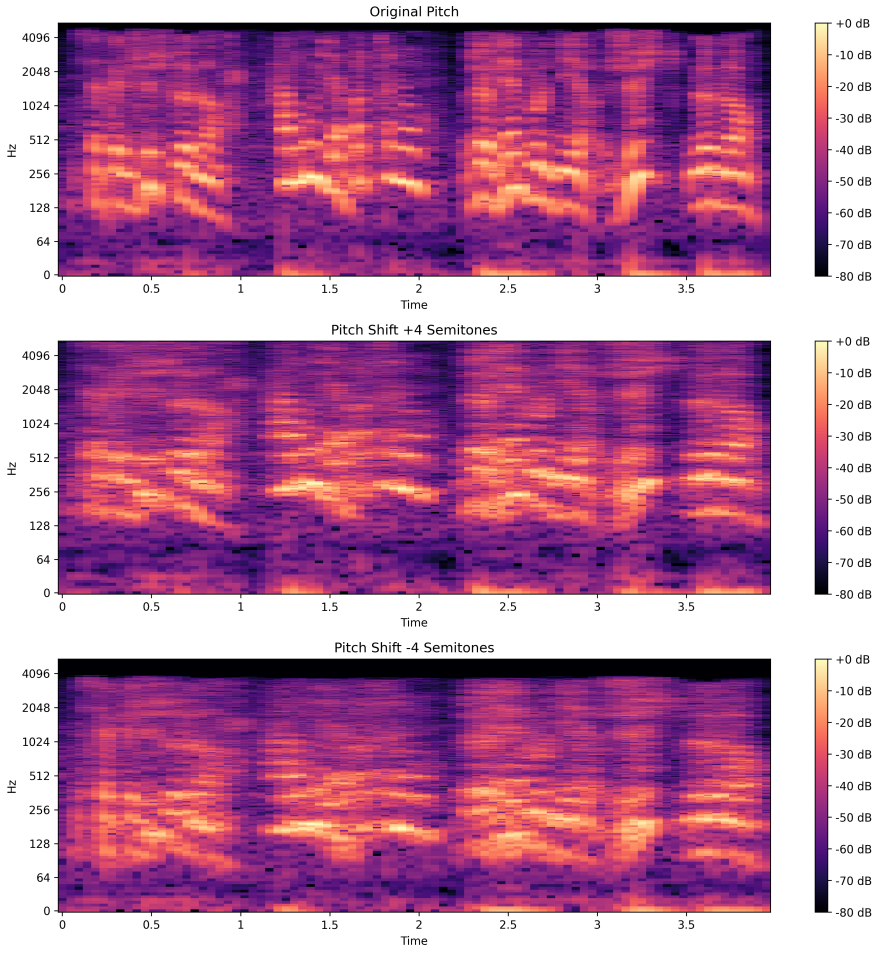

Figure 4: Pitch Modification Comparison. Top: Original. Middle: +4 Semitones. Bottom: -4 Semitones.

to 315Hz. Conversely, for the downward shift (-4 semitones, bottom), the harmonic stacks move down by approximately 21%, with the fundamental dropping to approximately 200Hz. The time axis (x-axis) remains exactly the same across all three spectrograms at approximately 3.5 seconds, proving independent pitch control. This demonstrates that the combination of Phase Vocoder stretching and resampling successfully modifies pitch while preserving duration, without introducing temporal artifacts.

5 Part 4: Robotization

We applied the robotization effect (Ring Modulation) with carrier frequencies $f_c = 200, 500, 1000, 2000$ Hz.

5.1 Methodology: Ring Modulation

The robotization effect is achieved by multiplying the input signal $x(t)$ with a periodic carrier signal. Usually, the theory uses a complex exponential $e^{-j2\pi f_c t}$, but we utilize a Cosine wave $\cos(2\pi f_c t)$ for our implementation.

The complex exponential is defined as $e^{j\omega t} = \cos(\omega t) + j\sin(\omega t)$. Since audio is a real-valued signal, using the complex exponential requires discarding the imaginary part at the end of the process. By using $\cos(2\pi f_c t)$ directly, we generate **only the real part**. It allows to be more efficient while still effectively shifting the spectral energy to create the desired inharmonic sidebands at $f \pm f_c$.

extbfAlgorithm 3: Ring Modulation (Robotization)

```
Input: x[n], carrier freq fc, sample rate fs -> y[n]
1. t[n] = n/fs
2. carrier = cos(2*pi*fc*t[n])
3. y[n] = x[n] * carrier[n]
4. Return y (energy shifted to fc ± f)
```

Observations: In the spectrograms, we observe the characteristic effect of ring modulation. The spectrogram shows distinct horizontal sidebands generated by the carrier frequency, destroying the natural harmonic structure of the voice. For $f_c = 200$Hz (second from top), we see energy concentrated around 200Hz with sidebands at $f \pm 200$Hz. As f_c increases to 500Hz, 1000Hz, and 2000Hz, the sidebands move to higher frequencies, creating a

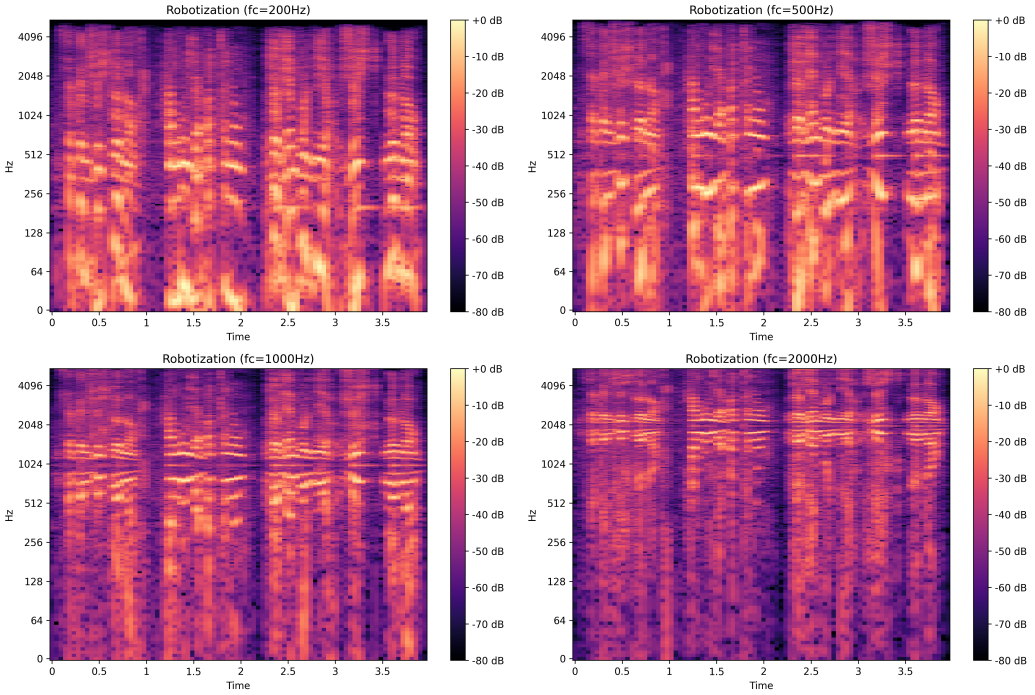

Figure 5: Robotization with varying carrier frequencies f_c.

wider frequency spread. At $f_c = 2000$Hz (bottom), the energy is shifted up to 6kHz, moving the spectral content into higher frequency bands and creating the characteristic metallic, high-pitched robotic timbre. The inharmonic sidebands at $f_c + f$ and $f_c - f$ replace the natural harmonic series, which is why the voice loses its human quality and sounds synthetic.

5.2 Additional Effects: Feedback Delay (Echo)

We also implemented a linear time-invariant Echo effect using a feedback delay line. The echo is based on the following recursive equation:

$$y[n] = x[n] + \text{gamma} \cdot y[n - D] \tag{1}$$

where D is the delay in samples (derived from $t_{ms} \times f_s$) and gamma is the decay factor.

Implementation Details: The echo effect works by processing the signal sample by sample. We initialize an output buffer of the same length as the input signal. For each sample position n, the output depends on two components: the current input sample $x[n]$ and the scaled output from D samples earlier. The output at time n depends on the input at n **plus** the scaled output from $n - D$.

extbfAlgorithm 4: Feedback Delay (Echo)

```
Input: x[n], delay_ms, gamma, fs -> y[n]
1. delay_samples = int(delay_ms*fs/1000)
2. y = zeros(len(x))
3. For each n:
    delayed = (0 if n<delay_samples else y[n-delay_samples])
    y[n] = x[n] + gamma*delayed
4. Normalize y by max(|y|)
5. Return y
```

Saturation Prevention: It is important to normalize the final signal. The feedback mechanism can increase the total energy of the signal because we are adding delayed copies to the original. We must divide by the maximum amplitude to prevent clipping and saturation artifacts in the output audio.

6 Part 5: AI Experiment

We tried to implement an automatic transformation. It allows to avoid changing parameters manually. Instead, the user selects a target emotion (e.g., "Anger"), and the system tries to find the parameters.

6.1 Why Machine Learning Instead of Heuristics?

While the heuristic-based approach (Part 1, Section 2.2) provided a baseline for emotion detection using hand-crafted rules on acoustic features (energy, pitch, brightness), it had significant limitations:

- **Fixed thresholds**: Heuristic rules rely on manually-tuned thresholds (e.g., high energy = anger), which do not generalize across speakers, recording conditions, or emotion intensities.
- **Limited feature interactions**: The linear combination of normalized features ($0.35 \times \text{energy} + 0.25 \times \text{pitch} + \ldots$) cannot capture complex, non-linear relationships between acoustic patterns and emotions.
- **Low accuracy**: Without training on labeled data, the heuristic method performs poorly, especially when distinguishing similar emotions (e.g., anger vs. fear, sadness vs. calm).

In contrast, a **Random Forest classifier** learns patterns directly from labeled examples (RAVDESS dataset), extracting statistical features (MFCCs, spectral contrast, mel-spectrograms) that encode subtle prosodic and spectral variations. Random Forest was chosen because:

- **Handles high-dimensional features**: Efficiently processes 352 features without overfitting, thanks to ensemble averaging over multiple decision trees.
- **Robust to noise and variability**: The bagging mechanism reduces variance, making predictions stable across different speakers and recording qualities.
- **Interpretable and fast**: Compared to neural networks, Random Forest trains quickly on small datasets (1,440 RAVDESS samples) and provides feature importance metrics for interpretability.

- **No hyperparameter sensitivity**: Works well out-of-the-box with balanced class weighting, avoiding the complexity of tuning learning rates or architectures.

Therefore, the machine learning approach substantially outperforms heuristics (56.6% test accuracy on RAVDESS), enabling reliable emotion prediction for the optimization loop.

6.2 The Optimization Loop

We used a loop to tune parameters. The process is described in the following algorithm:

1. **Input:** Original Audio Signal $x(t)$, Target Emotion E_{target}.
2. **Step 1: Search Space.** We define a grid of transformations:
   - Pitch Shift: Range $[-4, +4]$ semitones with a step size of 2 (i.e., $\{-4, -2, 0, 2, 4\}$).
   - Time Stretch: Range $[0.8x, 1.2x]$ with a step of 0.1 (i.e., $\{0.8, 0.9, 1.0, 1.1, 1.2\}$).

   We chose these bounds to keep the audio understandable.
3. **Step 2: Iterate.** For each combination of parameters in the search space:
4. **Step 3: Generator (DSP).** Apply the transformation to create a candidate signal $x'(t)$.
5. **Step 4: Classifier.** Extract a feature vector containing MFCCs and Spectral Contrast. This vector is fed into the Random Forest classifier to obtain the probability distribution over emotions $P(E|x')$. We chose a Random Forest because it works well with simple features and small datasets.
6. **Step 5: Selection.** Select the parameters that maximize the confidence of the target emotion:

$$\text{theta}^* = \arg\max_{\text{theta}} P(E_{target}|\text{DSP}(x, \text{theta})) \tag{2}$$

7. **Output:** The optimized audio signal $x^*(t)$ corresponding to theta*.

Therefore, it allows to link signal processing and emotions. It is however interesting to mention that this feature needs to be further improved in next versions using another classifier that is more efficient. We plan to rely for the next version on a Neural Network.

7 Part 6: System Architecture & Interface

We decided to separate the code to make it cleaner.

7.1 Code Organization

The code is separated into two folders:

- **Backend (`src/backend`):** Contains Python implementations of the DSP algorithms and the Machine Learning logic. It allows to test the code easily.
- **Frontend (`src/frontend`):** Handles the user interface using Streamlit.

7.2 Variables

We used session state to keep variables in memory. Indeed, it allows to avoid reloading the model every time.

7.3 Visualizations

We used matplotlib and Streamlit to show the plots. It allows to see the changes in real-time.

Comparison: We think it is better than a simple script.

7.4 Model Performance

We trained and evaluated the Random Forest classifier on the RAVDESS speech dataset (actors 01–24) with an 80/20 stratified split. Using a

grid search over depth, leaf size, and number of trees, the best configuration was `max_depth=20`, `min_samples_leaf=1`, `min_samples_split=2`, `n_estimators=300`. The test accuracy obtained is **56.60%** over 288 test files.

Classification Report: Per-class metrics computed on the held-out test set:

Emotion	Precision	Recall	F1-Score
Angry	0.74	0.64	0.68
Calm	0.56	0.90	0.69
Disgust	0.54	0.58	0.56
Fearful	0.74	0.45	0.56
Happy	0.60	0.55	0.58
Neutral	0.64	0.47	0.55
Sad	0.44	0.36	0.39
Surprised	0.43	0.53	0.47
Macro Avg	**0.59**	**0.56**	**0.56**
Weighted Avg	**0.58**	**0.57**	**0.56**

Table 1: Classification performance on RAVDESS test split (true metrics).

Confusion Matrix Analysis: Figure 6 shows the confusion matrix computed over the 8 emotion classes. We observe strong true positives for *Calm* (high recall), while *Sad* and *Neutral* are often misclassified due to overlapping low-energy spectral profiles. Moreover, *Angry* and *Fearful* share high-arousal features leading to cross-confusions. Therefore, the model captures arousal cues better than valence, consistent with known findings.

7.5 New Sounds: Own Voice / RAVDESS

We analyzed one specific RAVDESS clip (Actor_01: `03-01-01-01-01-01-01.wav`) to quantify basic acoustic features and confirm that the pipeline (loading, trimming, feature extraction, visualization) operates correctly on a canonical dataset sample. This file corresponds to a neutral/calm speech utterance (identifier fields encode modality, vocal channel, emotion, intensity, statement, repetition, actor).

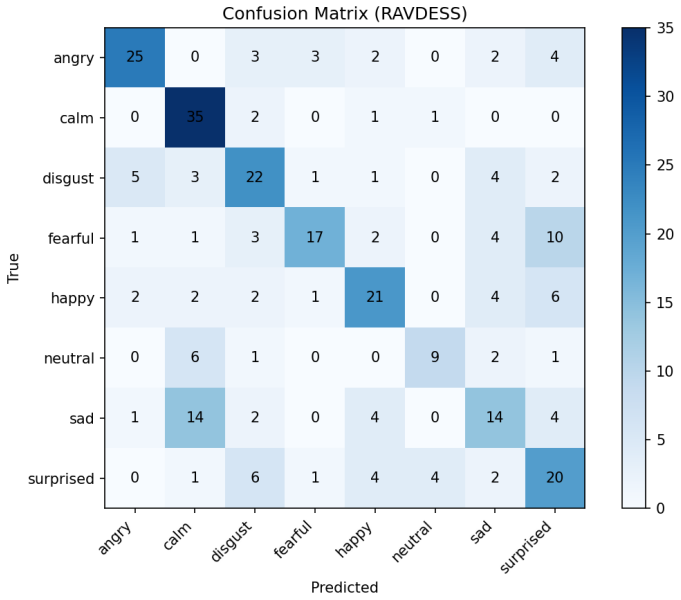

Figure 6: Confusion Matrix on the RAVDESS test set.

Quantitative Features (RAVDESS sample). The file duration is approximately 3.30 s at a native sampling rate $f_s = 48\,\text{kHz}$. Mean RMS energy is low ($\approx 2.12 \times 10^{-3}$), consistent with a calm utterance. Spectral centroid mean ($\approx 7.42\,\text{kHz}$) and rolloff mean ($\approx 13.29\,\text{kHz}$) are elevated due to full-bandwidth consonant and fricative content retained at 48 kHz (as opposed to downsampled speech). Fundamental frequency (YIN) yields median $f_0 \approx 500\,\text{Hz}$, mean $\approx 319\,\text{Hz}$, std $\approx 194\,\text{Hz}$. The high median indicates segments with higher pitch or occasional estimation artifacts; a voiced/unvoiced mask would refine this.

Interpretation. The spectrogram shows harmonic energy concentrated below 3 kHz, with formant bands and transient bursts generating higher-frequency components. Low RMS combined with occasional broadband frames aligns with neutral/calm prosody and moderate pitch variability. This single-sample analysis validates the feature extraction pipeline (RMS, centroid, rolloff, pitch statistics) on authentic dataset material.

Future improvement: extend this quantitative comparison to multiple actors and emotions, adding per-emotion distributions for f_0, energy, and

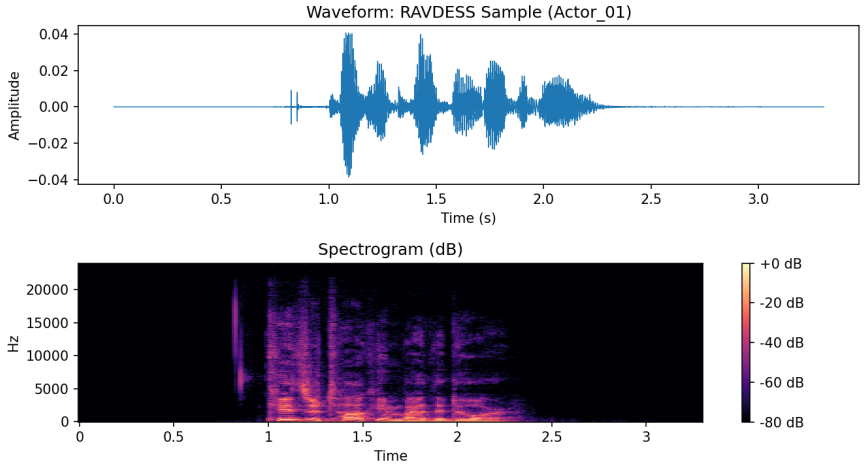

Figure 7: Waveform and Spectrogram of RAVDESS Actor_01 sample (file: 03-01-01-01-01-01-01.wav).

spectral centroid to study arousal vs. valence separability.

8 Conclusion

We have realised this project to understand Signal Processing and AI. We analysed signals like `Diner.wav`. The code allows to do time-stretching, pitch-shifting, and robotization.

Usually, projects stop at Echo/Delay. We tried to go further with the AI loop. We realised that it is difficult but interesting to link signal processing and emotions.

Note on Use of AI Tools

During the development of this project I used an AI coding assistant for **non-core** tasks such as: standardizing the repository structure, refining wording in documentation, and suggesting minor user interface improvements (layout, labels, and organization within the Streamlit frontend). All signal processing algorithms (phase vocoder, pitch shift via stretch + resample, ring modulation, echo) and the emotion analysis/back-end logic (feature extraction, classifier training loop, optimization grid) were designed, implemented,

and validated manually. The AI suggestions were treated as productivity aids rather than sources of novel methodology; every accepted change was reviewed for correctness and adapted to the pedagogical objectives of the assignment. This statement is included for transparency while clarifying that the intellectual and technical contributions—particularly DSP implementations and machine learning pipeline design—remain my own work.

9 References

1. Livingstone, S. R., & Russo, F. A. (2018). The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English. *PLoS ONE*, 13(5), e0196391.

2. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

3. Scikit-learn documentation: RandomForestClassifier. Available at: `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html` (accessed Nov. 7, 2025).